

Fish Length Estimation with Vision Foundation Models

The Complete A-to-Z Project Story

Shahman Butt · Area Seminar — Deep Learning for Maritime Vision Applications (VACOT), University of Rostock

Repository: github.com/Shahman-Butt/AreaSeminarFishLength

Every step is told twice, side by side: **in plain words** (left, blue) and **in technical detail** (right, gray).

The journey, in sequence

1. The assignment
2. Understanding the AutoFish paper
3. Setting up the workshop
4. Getting the data
5. Building the master index
6. The anti-cheating audit (split & leakage)
7. Cutting out the fish (preprocessing)
8. Defining success (metrics)
9. Designing the model
10. The training procedure
11. Experiment 1 — reproducing the baseline
12. What foundation models are
13. Experiments 2–5 — the DINOv2 saga
14. Experiments 6–7 — CLIP
15. Experiment 8 — ConvNeXt-Tiny
16. The final ranking & what it means
17. Good scientific practice built in
18. Deliverables & repository
19. What remains open
20. Glossary + full results appendix

1 The assignment

🔗 IN PLAIN WORDS

My professor gave me a two-part job. **Part one:** take a freshly published research paper about measuring fish from photos, rebuild its system from scratch, and prove I get the same result they published. **Part two:** once my rebuild is trustworthy, replace the "seeing" part of the system with newer, much larger AI models and find out — with a fair test — whether they measure fish better.

Why in that order? Because if I can't first match the published number, I can never know whether my later comparisons show real differences between models or just bugs in my own setup. Reproduction is the calibration of my whole laboratory.

⚙️ TECHNICAL DETAIL

Task 1 (Q1): reproduce the REG MobileNetV2 length-regression baseline from *Bengtson et al., "AutoFish: Dataset and Benchmark for Fine-grained Analysis of Fish", WACV Workshops 2025 (arXiv:2501.03767)*, targeting the reported test MAE of **0.62 cm** (non-occluded) and **1.38 cm** (occluded), on the official data split.

Task 2 (Q2): a controlled encoder-swap study: replace only the visual encoder with vision foundation models (DINOv2, CLIP) and a modern supervised encoder (ConvNeXt-Tiny), holding data, split, inputs, head, loss, and evaluation fixed, so that test-MAE differences are attributable to the encoder alone.

2 Understanding the AutoFish paper

🔍 IN PLAIN WORDS

Fisheries science runs on fish lengths: they reveal how old fish populations are, whether stocks are healthy, and whether catches respect minimum sizes. Today a human puts each fish on a measuring board — slow, costly, stressful for the fish. AutoFish is a benchmark for doing this with a camera instead.

The paper's authors photographed **454 real fish** from above on a white table, in **1,500 photos**, and hand-measured every fish. The photos come in an easy flavor (fish laid out separately) and a hard flavor (fish piled up, hiding each other — like on a real boat). Their length-guessing program, built on a small network called MobileNetV2, made an average mistake of **0.62 cm on the easy photos** and **1.38 cm on the hard ones**. Those two numbers are my targets.

⚙️ TECHNICAL DETAIL

Dataset: 1,500 top-view RGB images, 454 unique fish, 25 fish groups, 18,157 instance annotations (COCO-style: polygon segmentation, bbox, species, `fish_id`, hand-measured `length` in cm).
Published on Hugging Face as `vapaau/autofish`.

Image sets by file number: 00001–00020 = Set1, 00021–00040 = Set2 (both non-occluded singles), 00041–00060 = "All" (occluded piles). Each group was photographed across all sets.

Paper baseline ("REG"): MobileNetV2 encoder on masked fish crops + bbox input + MLP head, L1 regression to length; reported MAE 0.62 cm (Set1+Set2) / 1.38 cm (All). The official training release (Bitbucket) defines the group split I adopt.

3 Setting up the workshop

🔍 IN PLAIN WORDS

Training neural networks needs a powerful graphics card, which my laptop doesn't have. So I work in two places: I write and organize all code on my Windows laptop in VS Code, and the heavy computation runs on a university server that I control remotely over an encrypted connection (SSH). Think of it as designing at my desk and sending jobs to a factory.

On the server I created an isolated Python environment — a clean toolbox containing exactly the software versions I need — so results don't change because some unrelated software got updated. After every experiment, I copy the small result files back to my laptop; the huge model files stay on the server.

⚙️ TECHNICAL DETAIL

Local: Windows 11, VS Code, project at `Desktop/SEM3/AreaSeminar`, version-controlled with git.

Server: university GPU node (`baltic.informatik.uni-rostock.de`), NVIDIA RTX 5000 Ada, 32 GB VRAM; project mirrored at `~/autofish_baseline_repro`; Python 3.11 `venv` with PyTorch, Torchvision, OpenCLIP, pandas, NumPy, Pillow, scikit-learn, huggingface_hub, pycocotools (pinned in `requirements.txt`). Server default shell is tcsh, so commands are wrapped in `bash -lc`.

Transfer: code up / results down via `scp` with a dedicated SSH key. Per run, only `test_metrics.json`, `history.csv`, and the predictions CSV come back (~0.5 MB); checkpoints (~600 MB) remain on the server.

4 Getting the data

🔧 IN PLAIN WORDS

The dataset is publicly hosted on Hugging Face (a kind of GitHub for AI datasets and models). I wrote a small script that downloads the complete package — about 16 GB of photos plus one big annotation file that lists, for every fish in every photo: which fish it is, its species, exactly where it lies in the picture, its outline, and its true hand-measured length.

Using a script instead of clicking "download" matters: anyone can now fetch exactly the same data with one command, which is part of making the whole project repeatable.

⚙️ TECHNICAL DETAIL

`scripts/download_autofish.py` calls `huggingface_hub.snapshot_download(repo_id="vapaau/autofish", repo_type="dataset")` into `data/raw/autofish` — a complete, versioned snapshot.

Payload: 1,500 images (60 per group × 25 groups) plus `annotations.json` (COCO format): `images` (`id`, `file_name`, `group`, `width`, `height`), `categories` (`species`), `annotations` (`id`, `image_id`, `category_id`, `fish_id`, `length`, `bbox` [`x`,`y`,`w`,`h`], `polygon` `segmentation`, `side_up`).

5 Building the master index

🔧 IN PLAIN WORDS

The raw annotation file is one giant nested document — hard to work with directly. So my first real processing step converts it into a clean spreadsheet with **one row per fish-in-a-photo** (18,157 rows). Each row holds everything the rest of the project needs: which photo, which group, which set (easy or hard), the species, the fish's identity number, its rectangle and outline, and the true length.

This one table becomes the single source of truth. Every later step — cutting crops, training, evaluating — reads from it, so there is no possibility of two parts of the project silently disagreeing about the data.

⚙️ TECHNICAL DETAIL

`scripts/build_autofish_index.py` joins `annotations` × `images` × `categories` into `data/processed/index.csv` with columns: `annotation_id`, `image_id`, `file_name`, `image_path`, `group`, `split`, `set_name`, `species`, `fish_id`, `length_cm`, `bbox_x/y/w/h`, `segmentation`, `width`, `height`, `side_up`.

Two derived fields are computed here: `set_name` from the image number (1–20 Set1, 21–40 Set2, 41–60 All) and `split` from the group via the official split constant (Step 6). The script also writes `splits.json` (the split definition) and `exclusions.json` (the leakage log, Step 6) — machine-readable provenance for every data decision.

6 The anti-cheating audit — split & leakage

🗨️ IN PLAIN WORDS

To grade a model honestly, the final exam must contain only fish it has **never seen**. The trap: every physical fish appears in many photos. If photo A of a fish landed in the training pile and photo B of the same fish in the exam pile, the model could simply *remember that individual* — like recognizing a classmate rather than truly estimating size. This is called data leakage, and it silently inflates results.

The defense: fish are split by **group** — whole batches of fish go entirely to training, validation, or test. I additionally wrote an automatic check that verifies no fish identity crosses the boundary. It caught exactly one offender: fish number 113 appeared on both sides through a stray duplicate annotation. I removed the stray copy and the check now passes with zero leaks — and it re-runs every time the index is rebuilt, so leakage can never silently return.

⚙️ TECHNICAL DETAIL

Official group-level split (from the release):

train = {2,3,4,5,7,8,9,12,13,15,16,18,19,23,24} (15 groups)

val = {1,6,11,17,25} · test = {10,14,20,21,22} (5 each).

Leakage detection: group the index by `fish_id`, count distinct splits per fish; any count > 1 is a leak. Found: `fish_id 113`. **Resolution policy:** keep the group(s) where the fish occurs most; drop the singleton duplicate's annotations (logged by `annotation_id` in `exclusions.json`). The script then re-checks and hard-fails (`SystemExit`) if any leak remains.

Consequence: the non-occluded test subset has $n = 1,879$ instead of 1,880 — a documented, deliberate one-sample difference from the raw dataset. Validation $n = 3,639$; full test $n = 3,759$.

7 Cutting out the fish — preprocessing

🔗 IN PLAIN WORDS

The model never sees a whole table photo; it sees one fish at a time. For each of the 18,157 annotations I produce a small square picture of just that fish. Three refinements make these crops clean:

- **Everything that is not the fish is blacked out**, using the fish's hand-drawn outline. Even when another fish overlaps it in the photo, the crop shows only the fish being measured, on black.
- The cut-out window is made **square** around the fish's center, so that shrinking it later doesn't stretch the fish and distort its proportions — fatal for a length task.
- All crops are shrunk to the same size (224×224 pixels), the standard input size these networks expect.

Doing this once, up front, also makes training much faster: the expensive cutting work isn't repeated every epoch.

⚙️ TECHNICAL DETAIL

`scripts/make_crops.py` , per annotation:

1. Rasterize the `polygon` segmentation to a binary mask (`PIL ImageDraw.polygon`).
2. Paste the RGB image through the mask onto black — instance-masked image (`masking: "rgb"` in configs).
3. Compute the tight mask bbox, expand to a **square** of side `max(w,h)` about the mask centroid; crop with black padding where the square exits the image (aspect-ratio preserving).
4. Bilinear-resize to 224×224; save as `data/processed/crops/{annotation_id:06d}.png`.

Idempotent (skips existing files), verified complete by `scripts/check_processed.py` : 18,157 crops, 0 missing. Note the masking is what isolates the target fish under occlusion — the network sees the visible pixels of exactly one fish.

8 Defining success — the metrics

🗨 IN PLAIN WORDS

The headline number is beautifully simple: **on average, by how many centimetres is the model wrong?** Predict 30.5 cm for a 31 cm fish → error 0.5 cm. Average over all ~3,800 exam fish → the MAE. Smaller is better, and it's the same number the paper reports, so comparison is direct.

I track four helpers alongside: one that punishes big blunders extra hard (RMSE), the error as a percentage of fish size (MAPE), whether the model systematically guesses too long or too short (bias), and how much of the size variation it explains overall (R^2 , where 1.0 is perfect). And every result is always reported three ways: easy photos, hard photos, and both together.

⚙ TECHNICAL DETAIL

`src/autofish_vfm/metrics.py`, on $(y_{\text{true}}, y_{\text{pred}})$ in cm:

- **MAE** = $\text{mean } |\hat{y} - y|$ — primary, paper-comparable.
- **RMSE** = $\sqrt{\text{mean}((\hat{y} - y)^2)}$ — outlier-sensitive.
- **MAPE** = $\text{mean}(|\hat{y} - y|/y) \cdot 100$ — scale-relative.
- **Bias** = $\text{mean}(\hat{y} - y)$ — signed systematic offset.
- **R^2** = $1 - SS_{\text{res}}/SS_{\text{tot}}$.

Evaluation subsets (`evaluate.py`): `test_all` ($n=3,759$), `non_occluded_set1_set2` ($n=1,879$), `occluded_all` ($n=1,880$). Per-fish predictions are also dumped to `test_metrics.predictions.csv` (`annotation_id`, `fish_id`, `group`, `set`, `species`, `truth`, `prediction`) — the raw material for future error analysis.

9 Designing the model

🗨️ IN PLAIN WORDS

Every model in this project is the same three-part machine; only part one changes between experiments:

1. **The eye (encoder):** a pretrained network that turns the fish picture into a list of numbers summarizing what it sees. *This is the only part I swap.*
2. **The hint:** four numbers describing where and how big the fish's rectangle was in the original photo. Why? Because the crop was resized, the picture alone no longer says how large the fish really was — the rectangle restores the true scale. The paper's baseline uses this too.
3. **The decision-maker (head):** a small network that combines eye + hint into one number: length in cm.

Because parts two and three are identical everywhere, any difference in the final score is caused by the eye — which is precisely the research question.

⚙️ TECHNICAL DETAIL

`src/autofish_vfm/models.py` — shared contract:
`forward((image, bbox)) → length`. Encoder → feature vector, concatenated with `bbox = (x/W, y/H, w/W, h/H)`, fed to an MLP head of Linear→BatchNorm1d→ReLU blocks, final Linear→1.

Variant	Encoder / features	Dim	Head
Baseline	MobileNetV2, ImageNet-V1, classifier stripped	1280	[1000, 500, 1]
ConvNeXt	ConvNeXt-Tiny, ImageNet-V1, classifier→Identity	768	[512, 128, 1]
CLIP	OpenCLIP ViT-B/32 (openai), <code>encode_image</code>	512	[512, 128, 1]
DINOv2	<code>torch.hub dinov2_vits14</code> , CLS token	384	[512, 128, 1]

Adaptation modes (per config): *frozen* (encoder in eval mode, `requires_grad=False`, features under `no_grad`); *partial* (only the last transformer block + final norm/projection trainable); *full* (everything trainable, optionally with a separate encoder learning rate via optimizer param-groups).

10 The training procedure

🔧 IN PLAIN WORDS

Training is a loop: show the model a batch of fish, let it guess, measure how wrong it is, nudge its internal dials to be slightly less wrong, repeat. One pass through all ~11,000 training fish is an "epoch"; the baseline trains for 200 of them.

Three safeguards make this rigorous:

- **A referee set:** after every epoch the model is scored on validation fish it never trains on. The version that scores best there is the one kept — this prevents picking a version that merely memorized the training fish.
- **Color shuffling:** during training, each photo's colors are randomly jittered, teaching the model that lighting doesn't change a fish's length.
- **Smoke tests:** before any multi-hour run, a miniature 2-epoch version runs first to catch bugs cheaply. Every full run cost hours; every bug caught by a smoke test cost minutes.

And everything is written down: a fixed random seed, one config file per experiment listing every setting, and a per-epoch logbook.

⚙️ TECHNICAL DETAIL

`src/autofish_vfm/train_baseline.py` :

- Seeding: `random / numpy / torch(.cuda)` all seeded (42).
- Data: train loader shuffled, `ColorJitter(brightness .2, contrast .5, saturation .4, hue .3)` train-only; ImageNet normalization; `pin_memory` on CUDA.
- Optimization: L1 loss, Adam. Baseline: LR $1e-3$, batch 32, 200 epochs. Encoder swaps: LR $1e-4$, batch 16, 100 epochs. DINOv2 full fine-tune uses two param-groups: encoder LR $1e-5$ or $1e-6$, head LR $1e-4$.
- Checkpointing: `last.pt` every epoch; `best.pt` whenever validation MAE improves → model selection on val, never on test.
- Logging: per-epoch JSON line + full `history.csv` (train loss + all val metrics). Baseline history: val MAE 4.22 cm after epoch 1 → ~0.83 cm by epoch 200.
- Smoke configs (`*_smoke.json`) cap `max_train_batches / max_val_batches` and epochs for cheap end-to-end validation.
- Partial fine-tunes warm-start from the frozen run's best checkpoint (`resume_checkpoint`).

11 Experiment 1 — reproducing the baseline

🔍 IN PLAIN WORDS

The moment of truth: train MobileNetV2 exactly as described, evaluate on the untouched exam fish. **Paper: 0.62 cm. Me: 0.633 cm.** A tenth of a millimetre apart — reproduction successful. My entire laboratory is calibrated; every later comparison stands on this.

One honest surprise: on the *hard* overlapping-fish photos I scored **0.909 cm where the paper reports 1.38 cm** — noticeably better than the original. I don't hide this or claim credit for it; papers rarely specify every training detail, and mine likely differs somewhere (data augmentation, checkpoint choice, my leakage cleanup). It's flagged as an open question to investigate — that transparency *is* good scientific practice.

⚙️ TECHNICAL DETAIL

Config `baseline_official.json`: MobileNetV2, ImageNet-pretrained, full fine-tune, head [1000,500,1], L1, Adam 1e-3, batch 32, 200 epochs, 224², bbox input, seed 42. Run: `runs/baseline_official`.

Test subset	MAE	RMSE	MAPE	Bias	R ²	n
Non-occluded	0.633	1.027	1.96%	+0.080	0.965	1,879
Occluded	0.909	1.470	2.86%	−0.009	0.929	1,880
Full test	0.771	1.268	2.41%	+0.035	0.947	3,759

Δ vs paper: +0.013 cm non-occluded (≈2% relative) — within implementation noise. Occluded −0.47 cm vs paper (better); candidate explanations: unspecified augmentation/selection details in the paper, leakage cleanup. Bias ≈ 0 → no systematic over/underestimation; near-zero occluded bias with higher MAE = symmetric error inflation under occlusion, as expected from hidden fish extents.

12 What foundation models are — and the fairness rules

🔍 IN PLAIN WORDS

A **vision foundation model** is a giant network pretrained on enormous image collections until it acquires a broad, general "understanding" of pictures — learned once, reused everywhere. I tested two famous ones: **DINOv2** (Meta — learned from images alone, no labels, by teaching itself which views belong to the same image) and **CLIP** (OpenAI — learned from 400 million internet image–caption pairs, connecting pictures to words).

The analogy: MobileNetV2 is a tailor's apprentice I train for one job. A foundation model is a world-famous professor of seeing-in-general. Does the professor out-measure the trained apprentice?

Each giant gets three chances: **frozen** (use its knowledge as-is, train only the small decision-maker), **gently adapted** (unfreeze just its last layer), and **fully retrained**. And the fairness rule stands: same fish crops, same split, same hint, same decision-maker, same training budget class, same exam.

⚙️ TECHNICAL DETAIL

DINOv2 ViT-S/14 (via torch.hub): self-supervised (DINO/iBOT objectives), 21M-param vision transformer, 14×14 patches; I use the CLS token (384-d) as the global feature.

CLIP ViT-B/32 (OpenCLIP, "openai" weights): contrastive image–text pretraining on ~400M pairs; `encode_image` yields the 512-d projected embedding.

Controlled-variable design: identical index/crops/split, identical bbox input, same head family [512,128,1], L1 + Adam, 224² inputs, 100 epochs, seed 42. Adaptation ladder per encoder: frozen → last-block (warm-started from the frozen best checkpoint) → full fine-tune (DINOv2 only, with encoder-specific LR). Deliberately *no* per-encoder hyperparameter search — tuning effort would otherwise confound encoder quality (acknowledged as a design trade-off in Step 19).

13 Experiments 2–5 — the DINOv2 saga

🔍 IN PLAIN WORDS

Attempt 1 — frozen: 1.738 cm average error. More than double the baseline's 0.771. The professor, unadapted, is bad at this job.

Attempt 2 — full retraining (careful pace): 1.778 cm. Retraining everything didn't help; with only ~11,000 training fish, a transformer this size wobbles rather than converges nicely.

Attempt 3 — full retraining (ultra-careful pace): 2.132 cm — the worst result of the whole project. Fascinating failure: being *too* cautious meant the encoder changed just enough to disturb its pretrained knowledge, but not enough to learn the new task.

Attempt 4 — unfreeze only the last layer: 1.439 cm. Best DINOv2 result. Still nearly twice the baseline's error.

Verdict: across four escalating attempts, DINOv2 never came close. Its knowledge answers "*what* is this?"; our task asks "*exactly how big* is this?" — a different skill.

⚙️ TECHNICAL DETAIL

Run	Adaptation	Non-occ.	Occl.	Full
dinov2_vits14_frozen	head only	1.690	1.786	1.738
...finetune_lr1e5	full, enc LR 1e-5	1.636	1.919	1.778
...finetune_lr1e6	full, enc LR 1e-6	2.075	2.189	2.132
... lastblock_from_frozen	last block + norm	1.340	1.537	1.439

Reading: (a) CLS-token features are optimized for semantic invariance, discarding the fine metric-spatial information a mm-level regressor needs; (b) full ViT fine-tuning on ~11k samples is unstable — LR 1e-5 traded non-occluded gains for occluded degradation, LR 1e-6 underperformed even frozen features; (c) constrained adaptation (last block, warm-started) dominated both full-FT settings — a classic small-data transfer result.

14 Experiments 6–7 — CLIP, the surprise

🔍 IN PLAIN WORDS

Frozen CLIP:

1.002 cm —

massively better than frozen DINOv2 (1.738), with the identical setup. First real discovery: *not all giants are equal for this task*. CLIP grew up connecting images to words like "a large salmon" — plausibly forcing its features to keep more size-and-shape information. That's my hypothesis, stated as one.

Gently adapted

CLIP: 0.958 cm —

within touching distance of the baseline (0.771), but still behind. A foundation model got competitive; none got ahead.

⚙️ TECHNICAL DETAIL

Run	Adaptation	Non-occ.	Occl.	Full
clip_vitb32_frozen	head only	0.898	1.106	1.002
clip_vitb32_lastblock_from_frozen	last resblock + ln_post + proj	0.842	1.074	0.958

Best val MAE 1.003 cm at epoch 11 (early — the warm-started partial FT converges fast, then overfits mildly; best-checkpoint selection handles this). The frozen-CLIP vs frozen-DINOv2 gap (0.74 cm full-test) under identical protocol is the project's cleanest evidence that pretraining objective/data — not just architecture scale — governs transfer to metric regression. Mechanism unverified: would require representation-level probing (future work).

15 Experiment 8 — ConvNeXt-Tiny, the strongest challenger

IN PLAIN WORDS

The pattern so far — classic supervised training wins — suggested testing a *modern* network of the same school: ConvNeXt-Tiny (2022), trained on ImageNet like MobileNetV2 but with a decade-newer design, fully retrained for our task like the baseline was.

Result: 0.914 cm — best non-baseline model, beating both CLIPs. Yet the punchline holds: the humble MobileNetV2 stays number one at 0.771 cm. Even a newer, bigger network of the same family didn't dethrone it — at least not at this size and with this shared training recipe.

TECHNICAL DETAIL

Config `convnext_tiny_official.json`: ConvNeXt-Tiny (28M params vs MobileNetV2's 3.5M), ImageNet-V1, full FT, head [512,128,1], Adam 1e-4, batch 16, 100 epochs. Best val MAE 0.922 cm at epoch 83.

Subset	MAE	RMSE	MAPE	Bias	R ²
Non-occluded	0.814	1.181	2.50%	+0.136	0.954
Occluded	1.014	1.479	3.15%	+0.127	0.928
Full test	0.914	1.338	2.82%	+0.131	0.941

Caveat for fairness: ConvNeXt ran the swap-recipe (LR 1e-4, 100 ep), the baseline its paper recipe (LR 1e-3, 200 ep) — part of the remaining 0.14 cm gap could be recipe, not architecture. This is exactly what the planned multi-seed + recipe-matched reruns will resolve.

16 The final ranking & what it means

#	Model	Family	Non-occ. MAE	Occl. MAE	Full-test MAE
—	<i>Paper REG MobileNetV2 (reference)</i>	supervised CNN	0.62	1.38	—
1	MobileNetV2 (our reproduction)	supervised CNN	0.633	0.909	0.771
2	ConvNeXt-Tiny, full FT	supervised CNN	0.814	1.014	0.914
3	CLIP ViT-B/32, last-block FT	vision-language	0.842	1.074	0.958
4	CLIP ViT-B/32, frozen	vision-language	0.898	1.106	1.002
5	DINOv2 ViT-S/14, last-block FT	self-supervised	1.340	1.537	1.439
6	DINOv2 ViT-S/14, frozen	self-supervised	1.690	1.786	1.738
7	DINOv2, full FT (enc LR 1e-5)	self-supervised	1.636	1.919	1.778
8	DINOv2, full FT (enc LR 1e-6)	self-supervised	2.075	2.189	2.132

🔧 IN PLAIN WORDS

Three lessons, in ascending order of importance:

1. **Overlapping fish hurt every model** — the baseline's error grows ~44% from easy to hard photos. Occlusion is where real-world improvement is needed.
2. **Not all giants are equal:** CLIP transferred far better than DINOv2 under identical conditions — *how* a model was pretrained matters more than that it is big.
3. **Bigger and newer is not automatically better.** For a precision measuring task, a small network trained end-to-end for exactly this job beat every general-purpose giant we tried. General "understanding" of images and millimetre-precise measurement are different skills.

⚙️ TECHNICAL DETAIL

The ranking is perfectly stratified by family: fully fine-tuned supervised CNNs (ranks 1–2) > CLIP (3–4) > DINOv2 (5–8). Scoped claim: *at the model scales tested (ViT-S/14, ViT-B/32, Tiny), with a shared training budget and no per-encoder tuning, on this dataset*, task adaptation dominates pretraining pedigree for metric length regression. Family-level gaps (≥ 0.7 cm CNN $\leftrightarrow$ DINOv2) far exceed plausible seed noise; within-family gaps (0.14 cm MobileNetV2 $\leftrightarrow$ ConvNeXt) await multi-seed confirmation. Consistent secondary signal: all models degrade under occlusion (masked crops mitigate but can't restore hidden fish extents).

17 Good scientific practice, built in from the start

🔧 IN PLAIN WORDS

- **Repeatable:** one command re-downloads the data; one config file per experiment records every setting; fixed random seed; anyone can redo any run.
- **Honest:** the exam fish were never used for any decision; the one leakage case was found, fixed, and documented; the result that's *better* than the paper is reported as openly as the ones that are worse.
- **Fair:** all models compared under identical conditions, and where a condition differs (the baseline's own recipe), it's stated.
- **Disclosed:** AI tools assisted with code scaffolding and drafting; all experiments, numbers, and conclusions are my own and verified by me — disclosed per course policy.

⚙️ TECHNICAL DETAIL

- Provenance artifacts: `splits.json`, `exclusions.json`, `per-run config.json` (re-serialized into each run dir), `history.csv`, `test_metrics.json`, per-fish predictions CSV.
- Model selection strictly on validation MAE; test touched once per model.
- Environment pinned via `requirements.txt`; hardware documented.
- Leakage check is a hard assertion in the pipeline — it cannot regress silently.
- Threats to validity enumerated (single seed, no per-encoder tuning, small FM variants, shared bbox prior) rather than hidden.

18 Deliverables & repository

🗨️ IN PLAIN WORDS

Everything is public on GitHub: the code, all experiment configurations, all result files, the documentation, and the seminar poster. Only three things are excluded: the 16 GB dataset (one script re-downloads it), the huge trained-model files (they live on the server), and private access keys.

Also prepared: an A3 poster built around one main result figure, and a rehearsed one-minute presentation with prepared answers for the question round.

⚙️ TECHNICAL DETAIL

github.com/Shahman-Butt/AreaSeminarFishLength:

- `src/autofish_vfm/` — data, models, train, evaluate, metrics
- `scripts/` — download, index build (+leakage audit), crop generation, checks
- `configs/` — 14 experiment configs incl. smoke variants
- `runs/` — metrics, training histories, per-fish predictions for all 8 experiments
- `data/processed/` — index.csv, splits.json, exclusions.json
- `poster/` — A3 poster (print-ready HTML) + presentation prep
- Docs: README, full report, summary guide, handoff log

19 What remains open (say it before being asked)

🗨️ IN PLAIN WORDS

1. **Every result is one training run.** The huge gaps are certainly real; the small ones (baseline vs ConvNeXt) should be confirmed by repeating with three different random seeds.
2. **EfficientNet-B0 is next** — the logical next member of the winning supervised family.
3. **Error analysis:** which species, which sizes, which occlusion situations cause the biggest mistakes? The per-fish prediction files are already saved for exactly this.
4. **The occluded-result mystery** (mine better than the paper's) deserves a comparison against the official code.
5. Only small foundation-model variants were tested; larger ones (or smarter adaptation like LoRA) could change the picture — an honest boundary of my claims.

⚙️ TECHNICAL DETAIL

Planned protocol: (1) EfficientNet-B0, ImageNet, swap-recipe config for direct comparability; (2) 3-seed reruns of MobileNetV2 / ConvNeXt-Tiny / best EfficientNet → report mean $\pm$ std, assess whether the 0.14 cm gap exceeds seed variance; (3) stratified error analysis from the saved predictions CSVs (species $\times$ length-decile $\times$ set), worst-case gallery; (4) recipe-matched control (ConvNeXt at LR 1e-3/200ep or baseline at LR 1e-4/100ep) to separate architecture from recipe effects; (5) optional: DINOv2 with register tokens / larger CLIP, parameter-efficient fine-tuning.

Glossary (for the layman half of your brain)

Term	Meaning
Annotation	One fish in one photo, with its outline, rectangle, species, identity, and true length.
Bounding box (bbox)	The rectangle enclosing a fish in the original photo; its 4 numbers are the "scale hint".
CNN	Convolutional neural network — the classic image-network design (MobileNetV2, ConvNeXt).
Data leakage	Test information sneaking into training — here: the same fish on both sides of the split.
Encoder	The "eye": the network part that turns an image into a feature vector.
Epoch	One full pass through all training examples.
Fine-tuning	Continuing to train a pretrained network on the new task (fully or partially).
Foundation model	Very large network pretrained on massive data for general reuse (DINOv2, CLIP).
Frozen	Encoder weights locked; only the small head is trained.
MAE	Mean absolute error — average mistake in cm; the headline metric.
Occluded	Fish partially hidden by other fish (the "All" image set).
Regression head	The small final network producing the length number.
Seed	Fixed starting point for randomness, making runs repeatable.
Smoke test	Tiny cheap trial run to catch bugs before an expensive full run.
Validation set	Held-out referee data used to pick the best model version — distinct from the test set.
ViT	Vision transformer — the architecture underlying DINOv2 and CLIP's image side.

Appendix — every experiment at a glance (MAE cm)

Run directory	Encoder	Adaptation	LR	Ep.	Non-occ.	Occl.	Full
baseline_official	MobileNetV2	full FT	1e-3	200	0.633	0.909	0.771
convnext_tiny_official	ConvNeXt-Tiny	full FT	1e-4	100	0.814	1.014	0.914
clip_vitb32_lastblock_from_frozen	CLIP ViT-B/32	last block	1e-4	100	0.842	1.074	0.958
clip_vitb32_frozen	CLIP ViT-B/32	frozen	1e-4	100	0.898	1.106	1.002
dinov2_vits14_lastblock_from_frozen	DINOv2 ViT-S/14	last block	1e-4	100	1.340	1.537	1.439
dinov2_vits14_frozen	DINOv2 ViT-S/14	frozen	1e-4	100	1.690	1.786	1.738
dinov2_vits14_finetune_lr1e5	DINOv2 ViT-S/14	full FT	enc 1e-5	100	1.636	1.919	1.778
dinov2_vits14_finetune_lr1e6	DINOv2 ViT-S/14	full FT	enc 1e-6	100	2.075	2.189	2.132

The whole project in three sentences

I reproduced the AutoFish MobileNetV2 length-regression baseline to within 0.013 cm of the published result, validating my entire pipeline. I then swapped only the image encoder — four DINOv2 variants, two CLIP variants, and ConvNeXt-Tiny — under strictly identical conditions, and every alternative lost to the baseline: DINOv2 badly, CLIP narrowly, ConvNeXt closest. Based on current evidence, task-adapted supervised encoders beat general foundation features for precise fish length regression — with multi-seed validation, EfficientNet, and error analysis as the defined next steps.